

ORION Pipeline - Architecture Overview

AI Engineer Interview - AppliedAI 2026

1. Architecture and Data Flow

The pipeline is a sequential, serverless-compatible workflow composed of five discrete stages. Each stage is implemented as a standalone Python module, making the pipeline easy to test, swap, and audit independently.

- `ingest.py` - Loads the primary JSON submission and reads all referenced document files (TXT, PDF) from local or mounted object storage paths.
- `extract.py` - Sends the submission metadata and document contents to an LLM (GPT-4o-mini) via a structured prompt. Returns a typed risk profile covering eight dimensions: ownership, AML presence, regulatory history, cyber, financial health, privacy, PEP status, and criminal flags.
- `score.py` - Applies a deterministic weighted rubric to the extracted profile, producing per-dimension scores and a composite 0-10 risk score. Derives an authorization level (APPROVE / CONDITIONAL / DEFER / REJECT) via thresholds and hard rules for criminal flags and major regulatory history.
- `audit.py` - Appends a JSONL audit record per run containing a SHA-256 hash of all inputs, the model used, extracted profile, scores, and decision. Enables full reproducibility and regulatory traceability.
- `deliver.py` - POSTs the validated result payload to a configurable external review API endpoint (REVIEW_API_URL env var). Defaults to `httpbin` for testing.

Output is validated against a Pydantic schema (`schema.py`) before delivery, guaranteeing structural correctness and catching LLM outputs that contain out-of-vocabulary field values.

2. Key Technical and Modeling Decisions

- LLM for extraction, rules for scoring: The LLM handles reading unstructured documents and mapping them to a structured profile. Scoring is kept deterministic and auditable via a fixed rubric, so the authorization decision can always be explained without referencing model internals.
- Structured JSON output: The LLM is called with `response_format={type: json_object}`, eliminating markdown wrapping and parse failures.
- Input hashing for reproducibility: SHA-256 over sorted JSON of all inputs means any reviewer can verify that a given audit record corresponds to the exact document set that produced the decision.
- Pydantic validation as a safety net: Invalid LLM outputs (unknown enum values, missing required fields) raise a `ValidationError` immediately rather than propagating silently into the review API.
- Environment-driven configuration: Model, API keys, and review endpoint are injected via `.env` / environment variables. No secrets or URLs are hardcoded.

3. Trade-offs Under Execution and Accuracy Constraints

- Speed vs. accuracy: A single LLM call per submission keeps latency low (~4s) and fits within a serverless execution budget. Multi-call pipelines (e.g. per-document extraction then synthesis) would improve recall on large document sets but multiply cost and latency proportionally.
- Rule-based scoring vs. LLM scoring: Fixed weights are transparent and auditable but require manual calibration. Letting the LLM produce the composite score directly would adapt better to edge cases but sacrifices explainability. The hybrid approach (LLM extracts, rules score) balances both.
- Scoring calibration: The current rubric achieves 70% accuracy on the 20-case labeled dataset. The primary failure

mode is conflating DEFER and REJECT when multiple moderate risk factors co-occur. Accuracy improves iteratively as more labeled examples are added and weights recalibrated.

- Document format support: Only TXT and PDF are handled. DOCX and XLSX support can be added via python-docx and openpyxl without changing the pipeline contract.

4. Supporting Review, Audit and Iteration

- Human override: The structured payload exposes all intermediate signals (dimension scores, extracted profile, key findings) so a reviewer can identify exactly which factor drove a DEFER or REJECT and override the recommendation with full context.
- Audit trail: Every run writes a timestamped JSONL record to audit.jsonl. The input hash links any decision back to the exact document set that produced it, satisfying regulatory traceability requirements.
- Follow-up questions: The system generates non-duplicate, targeted clarification requests for each identified gap (missing AML policy, undisclosed UBOs, PEP due diligence). These are included in the reviewer payload and can be forwarded directly to the applicant.
- Iteration: Adding labeled cases to the dataset and re-running the batch evaluator immediately shows the impact on accuracy. Scoring weights and thresholds are isolated in score.py and can be tuned without touching extraction or delivery logic.